Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Lenguaje de Programación II

Guía Práctica — Sesión 7

Creación y arquitectura de la SPA: layout, navegación y CRUD de una tabla independiente

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	7 · 24 de septiembre de 2026
Modalidad	Individual, en laboratorio	Duración	2 horas prácticas
Tecnologías	Angular 22 · TypeScript · Spring Boot 4	Proyecto base	PharmaBackend (Unidad I)
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2

1. Situación de la sesión

En la Unidad I construiste PharmaBackend, la API REST que registra categorías, productos, clientes y ventas de una cadena de boticas. Hoy esa API solo se puede usar desde Swagger o Postman, herramientas pensadas para desarrolladores y no para el personal de la botica. El jefe de almacén necesita administrar el catálogo desde el navegador, con un menú claro, sin recargar la página en cada clic y viendo mensajes comprensibles cuando algo falla; por ejemplo, cuando intenta registrar una categoría que ya existe.

Tu tarea en esta unidad es construir **PharmaSoft**, la Single Page Application (SPA) que consume esa API. Esta sesión pone los cimientos: la estructura del proyecto, el marco visual (encabezado, menú bar y sidebar), la navegación entre módulos y el primer módulo completo sobre una tabla independiente: **Categorías**.

¿Cómo organizarías la SPA para que agregar un módulo nuevo no obligue a reescribir los que ya funcionan?

Reto 01 (actividad práctica, en clase): crear el proyecto frontend, construir su navegación principal y completar el CRUD de Categorías conectado a PharmaBackend.

Reto 02 (actividad autónoma): agregar el módulo Clientes con la misma arquitectura y revisar, en un informe técnico, la organización de módulos, rutas, menú y responsabilidades de componentes y servicios.

1.1 Resultado de aprendizaje y criterio que se trabaja

Elemento	Descripción
Resultado de aprendizaje de la unidad	Construye una SPA empresarial conectada al backend REST, con layout y navegación, CRUD de tablas independientes y dependientes, formularios cabecera–detalle, consultas y reportes; integra seguridad mediante JWT en el backend y gestión de sesión, guards e interceptores en el frontend.
Criterio 1 del producto	Crea la SPA con proyecto frontend, layout, menú bar, sidebar, encabezado, módulos, componentes, rutas, navegación, servicios HTTP y CRUD de una tabla independiente.

1.2 Requisitos previos

Verifica cada herramienta antes de empezar. Si alguna versión no coincide, avisa al docente antes del paso 1: la mayoría de los problemas de la sesión se originan aquí.

Herramienta	Versión mínima	Cómo verificarla
Node.js	22.22.3 o 24.15 (LTS)	`node -v`
Angular CLI	22.x	`ng version`
Git	2.40 o superior	`git --version`
Visual Studio Code	Con la extensión Angular Language Service	Panel de extensiones
PharmaBackend	Rama feature/consultas-reportes (o master actualizada), con Oracle en ejecución	Abrir `http://localhost:8080/swagger-ui.html`

2. Actividad práctica

Volvamos a la botica: el jefe de almacén espera una aplicación que pueda usar sin conocer Swagger. Con el Reto 01 construirás la base de PharmaSoft y el módulo de Categorías, paso a paso y con el acompañamiento del docente. Al terminar, la SPA leerá y modificará datos reales de tu base Oracle a través de PharmaBackend.

Campo	Contenido
Título	Proyecto frontend: SPA PharmaSoft con layout, navegación y CRUD de Categorías
Objetivo	Construye una SPA en Angular organizada en capas (core, shared, layout y features) que integra encabezado, menú bar, sidebar, rutas con carga diferida y un servicio HTTP, y relaciona cada operación del CRUD de Categorías con su endpoint de PharmaBackend, mostrando al usuario las respuestas de éxito y de error.
Duración	100 minutos de trabajo + 20 minutos de pruebas y retroalimentación

Paso 1. Actualizar PharmaBackend y verificar CORS

La SPA se ejecutará en `http://localhost:4200` y la API en `http://localhost:8080`. Son **orígenes distintos**, así que el navegador bloqueará las respuestas si el backend no autoriza explícitamente el origen de la SPA. La versión actualizada de PharmaBackend ya incluye esa autorización; descárgala en tu copia local:

```
cd PharmaBackend
git fetch origin
git checkout feature/consultas-reportes   # o master, cuando el docente la fusione
git pull
```

Revisa las dos piezas que habilitan CORS. La clase `CorsConfig` registra los orígenes permitidos para todas las rutas `/api/**`, y el perfil de desarrollo los declara uno por uno, nunca con el comodín `*`:

Archivo: `PharmaBackend/src/main/resources/application-dev.yaml (fragmento)`

```
app:
  cors:
    allowed-origins: http://localhost:5173,http://127.0.0.1:5173,http://localhost:4200
```

Archivo: `PharmaBackend/src/main/java/pe/edu/upeu/PharmaBackend/config/CorsConfig.java (fragmento)`

```
registry.addMapping("/api/**")
        .allowedOrigins(origenesPermitidos)
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("Content-Type", "Accept", "Authorization", "Origin")
        .exposedHeaders("Location")
        .allowCredentials(true)
        .maxAge(3600);
```

Cambio en la base de datos: Flyway

La versión actualizada crea el esquema con Flyway (`db/migration/V1__esquema_inicial.sql`) y Hibernate solo lo valida (`ddl-auto: validate`). Si tu esquema de la Unidad I ya existe, Flyway lo registra como versión 1 y no lo modifica. Si la API no arranca por un error de validación del esquema, avisa al docente antes de continuar.

Inicia la API y comprueba en Swagger que `GET /api/v1/categorias` devuelve datos. Si la tabla está vacía, registra dos o tres categorías desde Swagger para tener información que mostrar.

Paso 2. Crear el proyecto Angular

Abre una terminal en tu carpeta de trabajo (fuera del proyecto backend) y ejecuta los comandos siguientes. El CLI crea el proyecto, instala las dependencias y deja lista la configuración de rutas.

```
npm install -g @angular/cli
ng new pharma-frontend --routing --style=css --ssr=false
cd pharma-frontend
ng generate environments
ng serve -o
```

`--ssr=false` indica que la aplicación se ejecuta solo en el navegador, `ng generate environments` crea los archivos de entorno donde guardarás la URL de la API y `ng serve -o` abre la aplicación en `http://localhost:4200`. Detén el servidor con Ctrl + C antes de continuar con el paso 3.

Angular 22 y los nombres de archivo

Desde Angular 20, el CLI genera archivos sin sufijo (`header.ts` con la clase `Header`) y, desde Angular 22, los servicios usan el decorador `@Service()`. Si tu CLI es de una versión anterior verás `header.component.ts`, `HeaderComponent` y `@Injectable({ providedIn: 'root' })`: el funcionamiento es el mismo, pero ajusta los nombres en los `import`.

Paso 3. Generar la estructura por capas

Organizarás el código en cuatro carpetas con responsabilidades distintas. **core** guarda piezas únicas para toda la aplicación (configuración del menú, modelos comunes y utilidades); **shared**, componentes y páginas reutilizables; **layout**, el marco visual, y **features**, un módulo por tabla o proceso del negocio. Genera los componentes y el servicio con el CLI:

```
ng g c layout/main-layout
ng g c layout/header
ng g c layout/sidebar
ng g c features/inicio
ng g c features/categorias/pages/categoria-list
ng g c features/categorias/pages/categoria-form
ng g c shared/pages/no-encontrado
ng g s features/categorias/services/categoria-service
ng g interface features/categorias/models/categoria --type=model
```

Crea a mano las carpetas `core/config`, `core/models` y `core/utils`. Al terminar el paso 10, la carpeta `src` debe verse así:

```
src/
├── app/
│   ├── core/
│   │   ├── config/menu.ts
│   │   ├── models/error-response.ts
│   │   └── utils/http-error.ts
│   ├── shared/pages/no-encontrado/
│   ├── layout/
│   │   ├── main-layout/   header/   sidebar/
│   ├── features/
│   │   ├── inicio/
│   │   └── categorias/
│   │       ├── models/categoria.model.ts
│   │       ├── services/categoria-service.ts
│   │       ├── pages/categoria-list/
│   │       ├── pages/categoria-form/
│   │       └── categorias.routes.ts
│   ├── app.config.ts
│   ├── app.routes.ts
│   └── app.ts
├── environments/environment.ts, environment.development.ts
└── styles.css
```

Paso 4. Configurar el entorno y los proveedores

La URL base de la API se guarda en los archivos de entorno para no repetirla en cada servicio. Escribe el mismo contenido en `environment.ts` y en `environment.development.ts`.

Archivo: `src/environments/environment.ts`

```ts
export const environment = {
  apiUrl: 'http://localhost:8080/api/v1',
};
```

En `app.config.ts` se registran los proveedores globales. `withComponentInputBinding()` permite recibir el parámetro `:id` de la ruta como un `input()` del componente, y `provideHttpClient()` es el punto donde, en la sesión 11, agregarás el interceptor que envía el token JWT.

Archivo: `src/app/app.config.ts`

```ts
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(),
  ],
};
```

Paso 5. Modelos y manejo de errores comunes

Los modelos describen en TypeScript la forma exacta del JSON que envía y recibe PharmaBackend. `Categoria` corresponde a `CategoriaResponseDTO` y `CategoriaRequest`, a `CategoriaRequestDTO`.

Archivo: `src/app/features/categorias/models/categoria.model.ts`

```
export interface Categoria {
  id: number;
  nombre: string;
  descripcion: string | null;
  estado: boolean;
  fechaCreacion: string;
  fechaModificacion: string | null;
}

export interface CategoriaRequest {
  nombre: string;
  descripcion: string | null;
  estado: boolean;
}
```

`ErrorResponse` refleja el `ErrorResponseDTO` del `GlobalExceptionHandler` del backend. Las funciones de `http-error.ts` convierten cualquier error HTTP en un mensaje legible y extraen los errores de validación por campo; así ningún componente repite esa lógica.

Archivo: `src/app/core/models/error-response.ts`

```
export interface ErrorResponse {
  timestamp: string;
  status: number;
  error: string;
  message: string;
  path: string;
  validationErrors: Record<string, string> | null;
}
```

Archivo: `src/app/core/utils/http-error.ts`

```
import { HttpErrorResponse } from '@angular/common/http';
import { ErrorResponse } from '../models/error-response';

export function mensajeError(err: HttpErrorResponse): string {
  if (err.status === 0) {
    return 'No se pudo conectar con el servidor. Verifique que el backend esté en ejecución y que CORS
permita este origen.';
  }
  const cuerpo = err.error as ErrorResponse | null;
  return cuerpo?.message ?? `Error ${err.status}: ${err.statusText}`;
}

export function erroresDeValidacion(err: HttpErrorResponse): Record<string, string> {
  const cuerpo = err.error as ErrorResponse | null;
  return cuerpo?.validationErrors ?? {};
}
```

Paso 6. Servicio HTTP de Categorías

El servicio es el **único** lugar de la aplicación que conoce las URL de la API. Cada método devuelve un `Observable`: la petición no se envía hasta que un componente se suscribe con `subscribe()`.

Archivo: `src/app/features/categorias/services/categoria-service.ts`

```
import { inject, Service } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../../../environments/environment';
import { Categoria, CategoriaRequest } from '../models/categoria.model';

@Service()
export class CategoriaService {
  private readonly http = inject(HttpClient);
  private readonly url = `${environment.apiUrl}/categorias`;

  listar(): Observable<Categoria[]> {
    return this.http.get<Categoria[]>(this.url);
  }

  obtener(id: number): Observable<Categoria> {
    return this.http.get<Categoria>(`${this.url}/${id}`);
  }

  crear(dto: CategoriaRequest): Observable<Categoria> {
    return this.http.post<Categoria>(this.url, dto);
  }

  actualizar(id: number, dto: CategoriaRequest): Observable<Categoria> {
    return this.http.put<Categoria>(`${this.url}/${id}`, dto);
  }

  eliminar(id: number): Observable<void> {
    return this.http.delete<void>(`${this.url}/${id}`);
  }
}
```

Paso 7. Layout: encabezado, menú bar y sidebar

El menú se define en un solo arreglo. El sidebar lo recorre con `@for`, de modo que agregar un módulo nuevo exige una sola línea. En la sesión 11 filtrarás este arreglo según los permisos del usuario.

Archivo: `src/app/core/config/menu.ts`

```
export interface MenuItem {
  etiqueta: string;
  ruta: string;
  icono: string;
}

export const MENU: MenuItem[] = [
  { etiqueta: 'Inicio', ruta: '/inicio', icono: '🏠' },
  { etiqueta: 'Categorías', ruta: '/categorias', icono: '🗂' },
  // Actividad autónoma: { etiqueta: 'Clientes', ruta: '/clientes', icono: '👥' },
];
```

El **encabezado** contiene la marca, el menú bar con accesos generales y el botón ☰. Ese botón no oculta el menú por sí mismo: emite un evento con `output()` para que el layout decida.

Archivo: `src/app/layout/header/header.ts`

```
import { Component, output } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-header',
  imports: [RouterLink],
  templateUrl: './header.html',
  styleUrl: './header.css',
})
export class Header {
  readonly toggleMenu = output<void>();
}
```

Archivo: `src/app/layout/header/header.html`

```
<header class="header">
  <button type="button" class="btn-menu" aria-label="Mostrar u ocultar menú"
(click)="toggleMenu.emit()">☰</button>
  <a routerLink="/inicio" class="marca">PharmaSoft</a>
  <nav class="menubar">
    <a routerLink="/inicio">Inicio</a>
    <a href="http://localhost:8080/swagger-ui.html" target="_blank" rel="noopener">API</a>
  </nav>
  <span class="usuario">Invitado</span>
</header>
```

Archivo: `src/app/layout/header/header.css`

```
:host { display: block; }
.header {
  height: 64px; display: flex; align-items: center; gap: 16px;
  padding: 0 20px; background: #0f2e5c; color: #fff;
}
.btn-menu { background: none; border: 0; color: inherit; font-size: 22px; cursor: pointer; }
.marca { color: #f2a900; font-weight: 700; font-size: 20px; text-decoration: none; }
.menubar { display: flex; gap: 16px; margin-left: 24px; }
.menubar a { color: #dbe4f3; text-decoration: none; }
.menubar a:hover { color: #fff; }
.usuario { margin-left: auto; font-size: 14px; opacity: 0.85; }
```

El **sidebar** muestra los módulos del menú y resalta la opción activa con `routerLinkActive`.

Archivo: `src/app/layout/sidebar/sidebar.ts`

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';
import { MENU } from '../../core/config/menu';

@Component({
  selector: 'app-sidebar',
  imports: [RouterLink, RouterLinkActive],
  templateUrl: './sidebar.html',
  styleUrl: './sidebar.css',
})
export class Sidebar {
  protected readonly menu = MENU;
}
```

Archivo: `src/app/layout/sidebar/sidebar.html`

```
<nav class="sidebar">
  <p class="titulo">Módulos</p>
  @for (item of menu; track item.ruta) {
    <a [routerLink]="item.ruta" routerLinkActive="activo">
      <span aria-hidden="true">{{ item.icono }}</span> {{ item.etiqueta }}
    </a>
  }
</nav>
```

Archivo: `src/app/layout/sidebar/sidebar.css`

```
:host { display: block; }
.sidebar { height: 100%; min-width: 240px; padding: 16px 12px; background: #fff; border-right: 1px solid
#e3e8f0; }
.titulo { font-size: 12px; text-transform: uppercase; color: #6b7280; margin: 0 8px 8px; }
a { display: block; padding: 10px 12px; border-radius: 8px; color: #1f2937; text-decoration: none; }
a:hover { background: #eef2f8; }
a.activo { background: #0f2e5c; color: #fff; }
```

El **MainLayout** une las tres zonas con CSS Grid. El `<router-outlet />` interno es el único lugar donde cambia el contenido al navegar; el encabezado y el sidebar permanecen.

Archivo: `src/app/layout/main-layout/main-layout.ts`

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Header } from '../header/header';
import { Sidebar } from '../sidebar/sidebar';

@Component({
  selector: 'app-main-layout',
  imports: [RouterOutlet, Header, Sidebar],
  templateUrl: './main-layout.html',
  styleUrl: './main-layout.css',
})
export class MainLayout {
  protected readonly menuColapsado = signal(false);

  alternarMenu(): void {
    this.menuColapsado.update(valor => !valor);
  }
}
```

Archivo: `src/app/layout/main-layout/main-layout.html`

```
<div class="shell" [class.colapsado]="menuColapsado()">
  <app-header class="area-header" (toggleMenu)="alternarMenu()" />
  <app-sidebar class="area-sidebar" />
  <main class="area-main">
    <router-outlet />
  </main>
</div>
```

Archivo: `src/app/layout/main-layout/main-layout.css`

```
.shell {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: 64px 1fr;
  grid-template-areas:
    'header header'
    'sidebar main';
  min-height: 100vh;
  transition: grid-template-columns 0.2s ease;
}
.shell.colapsado {
  grid-template-columns: 0 1fr;
}
.area-header { grid-area: header; }
.area-sidebar { grid-area: sidebar; overflow: hidden; }
.area-main { grid-area: main; padding: 24px; background: #f4f6fa; }
```

Reemplaza el contenido de `styles.css` con los estilos globales de tarjetas, botones, tablas y alertas que comparten todas las páginas:

Archivo: `src/styles.css`

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, 'Segoe UI', Roboto, sans-serif; color: #1f2937; }
h2 { margin-top: 0; color: #0f2e5c; }
.card { background: #fff; border-radius: 12px; padding: 24px; box-shadow: 0 1px 3px rgb(0 0 0 / 0.08); }
.page-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 16px; }
.input { width: 100%; padding: 8px 10px; border: 1px solid #cbd5e1; border-radius: 8px; font: inherit; }
.btn { display: inline-block; padding: 8px 14px; border: 1px solid #cbd5e1; border-radius: 8px;
       background: #fff; color: #1f2937; text-decoration: none; font: inherit; cursor: pointer; }
.btn-primary { background: #0f2e5c; border-color: #0f2e5c; color: #fff; }
.btn-peligro { border-color: #b42318; color: #b42318; }
.btn:disabled { opacity: 0.6; cursor: not-allowed; }
.alerta { padding: 12px 16px; margin-bottom: 16px; border-radius: 8px; background: #fef3f2; color: #b42318; }
.tabla { width: 100%; border-collapse: collapse; background: #fff; border-radius: 12px; overflow: hidden; }
.tabla th { background: #0f2e5c; color: #fff; text-align: left; padding: 10px 12px; }
.tabla td { padding: 10px 12px; border-bottom: 1px solid #e3e8f0; }
.badge { padding: 2px 10px; border-radius: 999px; background: #dcfce7; color: #166534; font-size: 13px; }
.badge.inactivo { background: #f1f5f9; color: #64748b; }
```

Paso 8. Rutas y navegación

El componente raíz ya no dibuja nada: solo aloja el router. Reemplaza `app.ts` y elimina `app.html` y `app.css`.

Archivo: `src/app/app.ts`

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  template: '<router-outlet />',
})
export class App {}
```

Las rutas principales usan **MainLayout** como ruta padre. Cada feature se carga de forma diferida (`loadComponent` o `loadChildren`): su código se descarga solo cuando el usuario entra al módulo. La ruta `**` atrapa cualquier dirección inexistente.

Archivo: `src/app/app.routes.ts`

```
import { Routes } from '@angular/router';
import { MainLayout } from './layout/main-layout/main-layout';

export const routes: Routes = [
  {
    path: '',
    component: MainLayout,
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'inicio' },
      {
        path: 'inicio',
        title: 'Inicio',
        loadComponent: () => import('./features/inicio/inicio').then(m => m.Inicio),
      },
      {
        path: 'categorias',
        loadChildren: () =>
          import('./features/categorias/categorias.routes').then(m => m.CATEGORIAS_ROUTES),
      },
    ],
  },
  {
    path: '**',
    title: 'Página no encontrada',
    loadComponent: () =>
      import('./shared/pages/no-encontrado/no-encontrado').then(m => m.NoEncontrado),
  },
];
```

Cada feature declara sus propias rutas. El mismo formulario atiende el registro (nuevo) y la edición (`:id/editar`).

Archivo: `src/app/features/categorias/categorias.routes.ts`

```
import { Routes } from '@angular/router';
import { CategoriaList } from './pages/categoria-list/categoria-list';
import { CategoriaForm } from './pages/categoria-form/categoria-form';

export const CATEGORIAS_ROUTES: Routes = [
  { path: '', component: CategoriaList, title: 'Categorías' },
  { path: 'nuevo', component: CategoriaForm, title: 'Nueva categoría' },
  { path: ':id/editar', component: CategoriaForm, title: 'Editar categoría' },
];
```

Completa las páginas de inicio y de error 404:

Archivo: `src/app/features/inicio/inicio.ts`

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-inicio',
  imports: [RouterLink],
  templateUrl: './inicio.html',
})
export class Inicio {}
```

Archivo: `src/app/features/inicio/inicio.html`

```
<section class="card">
  <h2>Bienvenido a PharmaSoft</h2>
  <p>Seleccione un módulo en el menú lateral para comenzar.</p>
  <a routerLink="/categorias" class="btn btn-primary">Ir a Categorías</a>
</section>
```

Archivo: `src/app/shared/pages/no-encontrado/no-encontrado.ts`

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-no-encontrado',
  imports: [RouterLink],
  template: `
    <section class="card">
      <h2>404 · Página no encontrada</h2>
      <a routerLink="/inicio" class="btn btn-primary">Volver al inicio</a>
    </section>
  `,
})
export class NoEncontrado {}
```

Elimina los archivos `inicio.css`, `no-encontrado.html` y `no-encontrado.css`, porque esos componentes no los usan. Ejecuta ng `serve -o`: ya debes poder navegar entre Inicio y la página 404 con el layout completo.

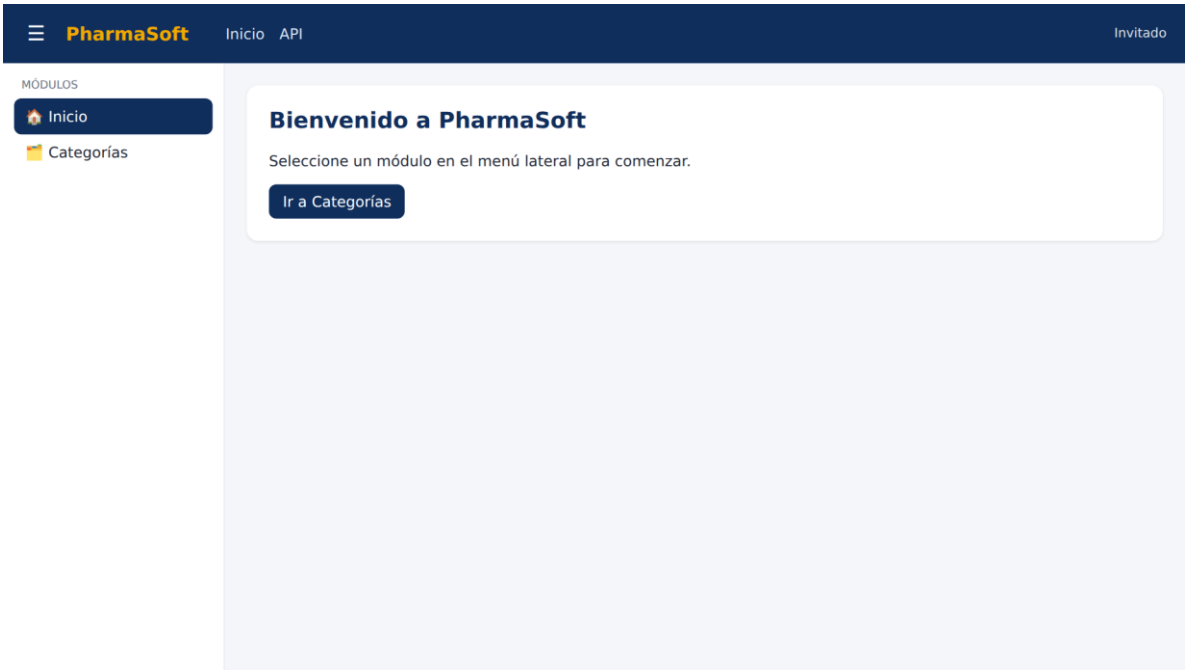


Figura 1. Layout de PharmaSoft con encabezado, menú bar y sidebar. Nota. Elaboración propia.

Paso 9. Listado de categorías

El listado guarda su estado en **signals**: `categorias` (los datos), `cargando`, `error` y `filtro`. `filtradas` es un `computed` que se recalcula solo cuando cambian los datos o el texto del buscador. Como Angular 22 funciona sin zone.js, la vista se actualiza porque los datos viven en signals.

Archivo: `src/app/features/categorias/pages/categoria-list/categoria-list.ts`

```
import { Component, computed, inject, OnInit, signal } from '@angular/core';
import { HttpErrorResponse } from '@angular/common/http';
```

```
import { RouterLink } from '@angular/router';
import { Categoria } from '../../models/categoria.model';
import { CategoriaService } from '../../services/categoria-service';
import { mensajeError } from '../../../../core/utils/http-error';

@Component({
  selector: 'app-categoria-list',
  imports: [RouterLink],
  templateUrl: './categoria-list.html',
  styleUrl: './categoria-list.css',
})
export class CategoriaList implements OnInit {
  private readonly categoriaService = inject(CategoriaService);

  protected readonly categorias = signal<Categoria[]>([]);
  protected readonly cargando = signal(false);
  protected readonly error = signal<string | null>(null);
  protected readonly filtro = signal('');

  protected readonly filtradas = computed(() => {
    const texto = this.filtro().trim().toLowerCase();
    return this.categorias().filter(c => c.nombre.toLowerCase().includes(texto));
  });

  ngOnInit(): void {
    this.cargar();
  }

  cargar(): void {
    this.cargando.set(true);
    this.error.set(null);
    this.categoriaService.listar().subscribe({
      next: datos => {
        this.categorias.set(datos);
        this.cargando.set(false);
      },
      error: (err: HttpErrorResponse) => {
        this.error.set(mensajeError(err));
        this.cargando.set(false);
      },
    });
  }

  eliminar(categoria: Categoria): void {
    if (!confirm(`¿Eliminar la categoría "${categoria.nombre}"?`)) {
      return;
    }
    this.categoriaService.eliminar(categoria.id).subscribe({
      next: () => this.categorias.update(lista => lista.filter(c => c.id !== categoria.id)),
      error: (err: HttpErrorResponse) => this.error.set(mensajeError(err)),
    });
  }
}
```

La plantilla usa el control de flujo `@if`, `@for` y `@empty`. Observa que `track c.id` le indica a Angular cómo identificar cada fila para no volver a dibujar toda la tabla.

Archivo: `src/app/features/categorias/pages/categoria-list/categoria-list.html`

```
<section class="page">
  <header class="page-header">
    <h2>Categorías</h2>
    <a routerLink="nuevo" class="btn btn-primary">+ Nueva categoría</a>
  </header>

  <input class="input buscador" placeholder="Buscar por nombre…"
         [value]="filtro()" (input)="filtro.set($any($event.target).value)" />
```

```
  @if (error()) {
    <div class="alerta">{{ error() }}</div>
  }

  @if (cargando()) {
    <p>Cargando categorías…</p>
  } @else {
    <table class="tabla">
      <thead>
        <tr><th>ID</th><th>Nombre</th><th>Descripción</th><th>Estado</th><th>Acciones</th></tr>
      </thead>
      <tbody>
        @for (c of filtradas(); track c.id) {
          <tr>
            <td>{{ c.id }}</td>
            <td>{{ c.nombre }}</td>
            <td>{{ c.descripcion ?? '—' }}</td>
            <td>
              <span class="badge" [class.inactivo]="!c.estado">{{ c.estado ? 'Activo' : 'Inactivo'
}}</span>
            </td>
            <td class="acciones">
              <a [routerLink]="[c.id, 'editar']" class="btn">Editar</a>
              <button type="button" class="btn btn-peligro" (click)="eliminar(c)">Eliminar</button>
            </td>
          </tr>
        } @empty {
          <tr><td colspan="5" class="vacio">No hay categorías registradas.</td></tr>
        }
      </tbody>
    </table>
  }
</section>
```

Archivo: `src/app/features/categorias/pages/categoria-list/categoria-list.css`

```
.buscador { max-width: 320px; margin-bottom: 16px; }
.acciones { display: flex; gap: 8px; }
.vacio { text-align: center; color: #6b7280; padding: 24px; }
```

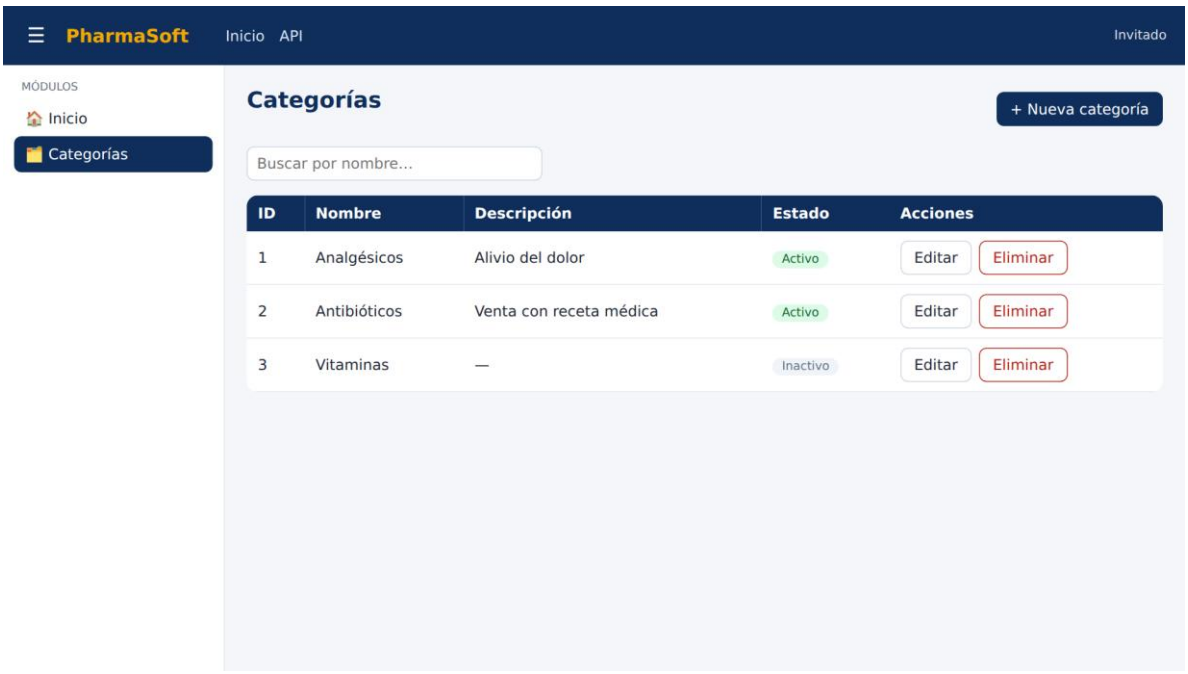


Figura 2. Listado de categorías obtenido desde PharmaBackend. Nota. Elaboración propia.

Paso 10. Formulario para registrar y editar

El formulario reactivo replica las reglas del `CategoriaRequestDTO` (nombre obligatorio de 3 a 50 caracteres y descripción de hasta 200). Estas validaciones mejoran la experiencia del usuario, pero **no reemplazan** las del backend: si la API responde 400 o 409, el formulario muestra su mensaje.

Archivo: `src/app/features/categorias/pages/categoria-form/categoria-form.ts`

```
import { Component, inject, input, OnInit, signal } from '@angular/core';
import { HttpErrorResponse } from '@angular/common/http';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { CategoriaRequest } from '../../models/categoria.model';
import { CategoriaService } from '../../services/categoria-service';
import { erroresDeValidacion, mensajeError } from '../../../../core/utils/http-error';

@Component({
  selector: 'app-categoria-form',
  imports: [ReactiveFormsModule, RouterLink],
  templateUrl: './categoria-form.html',
  styleUrl: './categoria-form.css',
})
export class CategoriaForm implements OnInit {
  private readonly fb = inject(NonNullableFormBuilder);
  private readonly categoriaService = inject(CategoriaService);
  private readonly router = inject(Router);

  /** Llega desde la ruta ':id/editar' gracias a withComponentInputBinding(). */
  readonly id = input<string>();

  protected readonly guardando = signal(false);
  protected readonly error = signal<string | null>(null);
  protected readonly erroresServidor = signal<Record<string, string>>({});

  protected readonly form = this.fb.group({
    nombre: ['', [Validators.required, Validators.minLength(3), Validators.maxLength(50)]],
    descripcion: ['', [Validators.maxLength(200)]],
    estado: [true],
  });

  protected esEdicion(): boolean {
    return !!this.id();
  }

  ngOnInit(): void {
    const id = this.id();
    if (id) {
      this.categoriaService.obtener(Number(id)).subscribe({
        next: c => this.form.setValue({
          nombre: c.nombre,
          descripcion: c.descripcion ?? '',
          estado: c.estado,
        }),
        error: (err: HttpErrorResponse) => this.error.set(mensajeError(err)),
      });
    }
  }

  guardar(): void {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
      return;
    }
    const valores = this.form.getRawValue();
    const dto: CategoriaRequest = {
      nombre: valores.nombre.trim(),
      descripcion: valores.descripcion.trim() || null,
      estado: valores.estado,
    };
    const id = this.id();
```

```
    const peticion = id
      ? this.categoriaService.actualizar(Number(id), dto)
      : this.categoriaService.crear(dto);

    this.guardando.set(true);
    peticion.subscribe({
      next: () => this.router.navigate(['/categorias']),
      error: (err: HttpErrorResponse) => {
        this.guardando.set(false);
        this.error.set(mensajeError(err));
        this.erroresServidor.set(erroresDeValidacion(err));
      },
    });
  }
}
```

Archivo: `src/app/features/categorias/pages/categoria-form/categoria-form.html`

```
<section class="page">
  <h2>{{ esEdicion() ? 'Editar categoría' : 'Nueva categoría' }}</h2>

  @if (error()) {
    <div class="alerta">{{ error() }}</div>
  }

  <form [formGroup]="form" (ngSubmit)="guardar()" class="card formulario">
    <label>
      Nombre
      <input class="input" formControlName="nombre" />
    </label>
    @if (form.controls.nombre.touched && form.controls.nombre.invalid) {
      <small class="error">El nombre es obligatorio y debe tener entre 3 y 50 caracteres.</small>
    }
    @if (erroresServidor()['nombre']; as mensaje) {
      <small class="error">{{ mensaje }}</small>
    }

    <label>
      Descripción
      <textarea class="input" rows="3" formControlName="descripcion"></textarea>
    </label>
    @if (form.controls.descripcion.invalid) {
      <small class="error">Máximo 200 caracteres.</small>
    }

    <label class="check">
      <input type="checkbox" formControlName="estado" /> Activo
    </label>

    <div class="acciones">
      <a routerLink="/categorias" class="btn">Cancelar</a>
      <button type="submit" class="btn btn-primary" [disabled]="guardando()">
        {{ guardando() ? 'Guardando…' : (esEdicion() ? 'Actualizar' : 'Registrar') }}
      </button>
    </div>
  </form>
</section>
```

Archivo: `src/app/features/categorias/pages/categoria-form/categoria-form.css`

```
.formulario { display: flex; flex-direction: column; gap: 10px; max-width: 520px; }
label { display: flex; flex-direction: column; gap: 4px; font-weight: 600; }
.check { flex-direction: row; align-items: center; gap: 8px; }
.error { color: #b42318; }
.acciones { display: flex; justify-content: flex-end; gap: 8px; margin-top: 8px; }
```

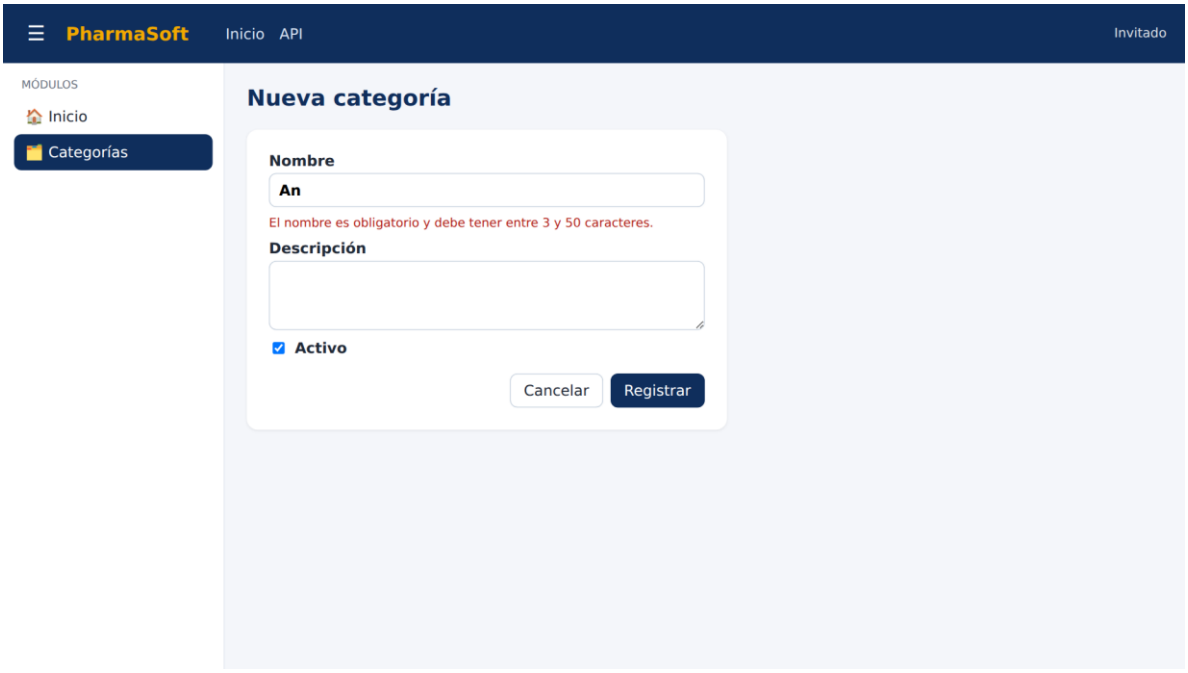


Figura 3. Validación del formulario antes de enviar la petición. Nota. Elaboración propia.

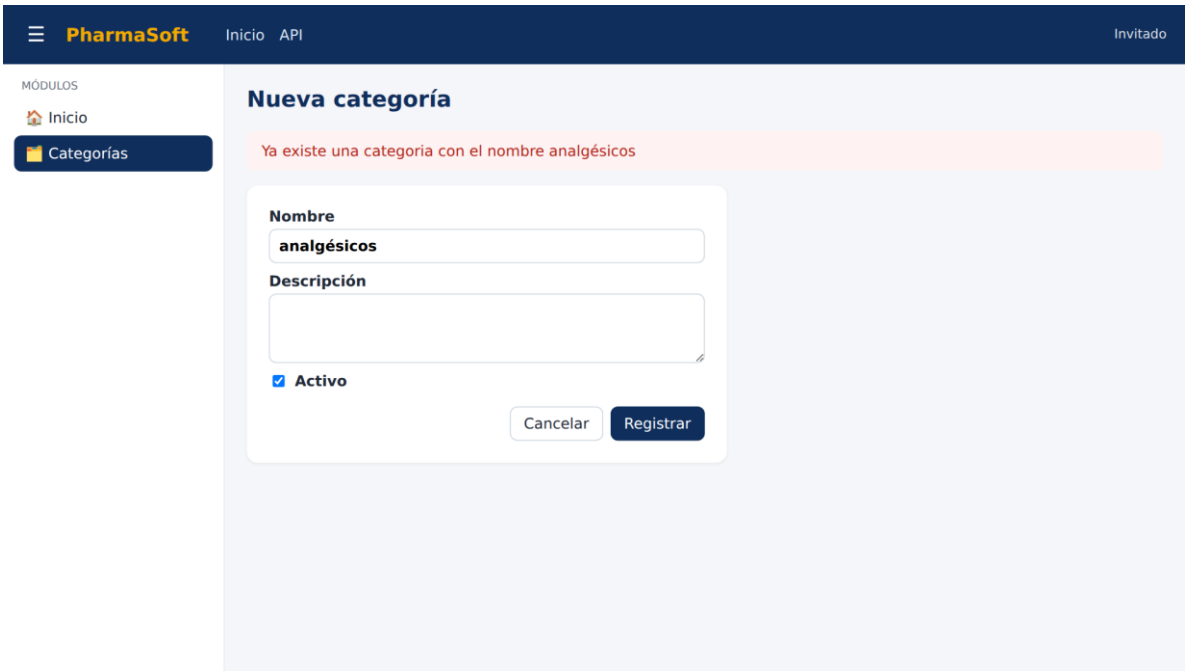


Figura 4. Respuesta 409 del backend mostrada al usuario. Nota. Elaboración propia.

Paso 11. Probar la SPA

Con PharmaBackend y `ng serve` en ejecución, abre las herramientas de desarrollador del navegador (F12) en la pestaña **Red** y ejecuta estas pruebas en orden. Registra una captura de cada una para tu evidencia.

N.º	Acción	Resultado esperado
1	Abrir `http://localhost:4200`	Redirige a /inicio y el sidebar resalta «Inicio».
2	Hacer clic en «Categorías»	Muestra las mismas categorías que Swagger (GET 200).
3	Escribir parte de un nombre en el buscador	La tabla se filtra sin enviar peticiones nuevas.

N.º	Acción	Resultado esperado
4	Registrar una categoría con el nombre «An»	Aparece el mensaje de validación y no se envía ninguna petición.
5	Registrar una categoría con un nombre ya existente	Alerta con el mensaje del backend (POST 409).
6	Registrar una categoría válida	Vuelve al listado con la nueva fila (POST 201).
7	Editar una categoría y desmarcar «Activo»	Formulario precargado; al guardar, la fila muestra «Inactivo» (PUT 200).
8	Eliminar una categoría que tiene productos	Alerta con el mensaje del backend (DELETE 409).
9	Eliminar una categoría sin productos	La fila desaparece (DELETE 204).
10	Detener PharmaBackend y recargar /categorias	Mensaje «No se pudo conectar con el servidor…».
11	Escribir `http://localhost:4200/xyz`	Página «404 · Página no encontrada».
12	Pulsar el botón ☰	El sidebar se oculta y se muestra sin recargar la página.

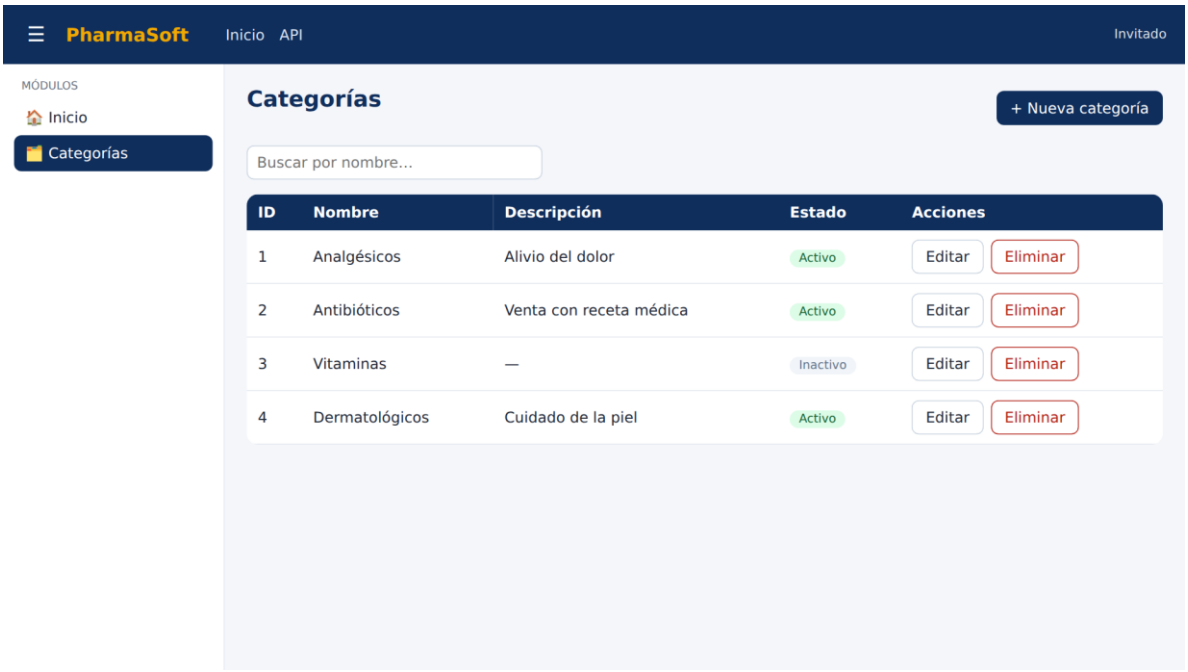


Figura 5. Resultado esperado: listado actualizado después de registrar una categoría. Nota. Elaboración propia.

Paso 12. Versionar el trabajo

Crea el repositorio `pharma-frontend` y trabaja en una rama propia. Haz un commit cada vez que una parte funcione; no esperes al final.

```
git init
git add .
git commit -m "chore: crear proyecto Angular pharma-frontend"
git branch -M main
git remote add origin https://github.com/<usuario>/pharma-frontend.git
git push -u origin main
git checkout -b develop && git push -u origin develop
git checkout -b feature/spa-categorias-<apellido>

git commit -am "feat(layout): crear encabezado, menú bar y sidebar"
git commit -am "feat(categorias): listar categorías desde la API"
git commit -am "feat(categorias): registrar y editar con formulario reactivo"
git commit -am "feat(categorias): eliminar con confirmación y mostrar errores"
git push -u origin feature/spa-categorias-<apellido>
```

En esta práctica no modificas PharmaBackend: solo lo consumes. Recuerda que `git commit -am` solo incluye archivos ya rastreados: después de crear archivos nuevos usa `git add .` antes del commit.

Producto esperado

- Repositorio `pharma-frontend` con la rama `feature/spa-categorias-<apellido>` y al menos cuatro commits que sigan la convención de mensajes.
- SPA en ejecución con layout (encabezado, menú bar y sidebar), rutas con carga diferida, página 404 y CRUD de Categorías conectado a PharmaBackend.
- Documento PDF `Apellido_LP2_S7_Practica.pdf` con las capturas de las doce pruebas del paso 11, mostrando la pestaña Red cuando corresponda.

Lista de cotejo

N.º	Criterio observable	Sí	No
1	El proyecto se crea con Angular CLI y compila con `ng build` sin errores.		
2	El código está organizado en core, shared, layout y features, con los archivos en la carpeta que les corresponde.		
3	La URL de la API se lee desde `environment` y no está escrita dentro de los componentes.		
4	El layout muestra encabezado, menú bar y sidebar, que permanecen fijos al navegar.		
5	El sidebar se construye desde `MENU` y resalta la opción activa.		
6	Las rutas usan MainLayout como padre, cargan las features de forma diferida y tienen una página 404.		
7	`CategoriaService` concentra las cinco operaciones HTTP; ningún componente usa `HttpClient` directamente.		
8	El listado muestra estados de carga, vacío y error, y el buscador filtra con un `computed`.		
9	El formulario registra y edita con validaciones equivalentes a las del DTO del backend.		
10	Los errores 400, 404 y 409 y la falta de conexión se muestran con el mensaje del backend o uno comprensible.		
11	La eliminación pide confirmación y actualiza la tabla sin recargar la página.		

N.º	Criterio observable	Sí	No
12	La SPA consume la versión actualizada de PharmaBackend sin errores de CORS en la consola del navegador.		
13	Existe la rama de funcionalidad con al menos cuatro commits que siguen la convención.		

3. Problemas frecuentes

Antes de pedir ayuda, revisa la consola del navegador (F12) y la terminal de `ng serve`: el mensaje exacto del error casi siempre indica la causa.

Síntoma	Causa probable	Solución
«The Angular CLI requires a minimum Node.js version…»	Versión de Node.js anterior a la que exige Angular 22.	Instala Node.js 24 LTS y vuelve a abrir la terminal.
«…has been blocked by CORS policy» en la consola	Tu copia de PharmaBackend no está actualizada o el origen no figura en `app.cors.allowed-origins`.	Haz `git pull` de la rama actualizada, verifica que figure `http://localhost:4200` (sin barra final) y reinicia la API.
Mensaje «No se pudo conectar con el servidor…»	La API está detenida o escucha en otro puerto (status 0).	Inicia PharmaBackend y verifica `apiUrl` en `environment.ts`.
NG8001: 'app-header' is not a known element	El componente no está en el arreglo `imports` de quien lo usa.	Agrega `Header` a `imports` de `MainLayout` e impórtalo arriba.
Can't bind to 'formGroup'	Falta `ReactiveFormsModule` en el componente del formulario.	Agrégalo al arreglo `imports` de `CategoriaForm`.
La tabla no se actualiza al llegar los datos	El dato se guardó en una variable normal y no en un signal.	Usa `signal()` y actualízalo con `set()` o `update()`.
Al editar, el formulario aparece vacío	Falta `withComponentInputBinding()` o el input no se llama `id`.	Revisa `app.config.ts` y que la ruta sea `:id/editar`.
Cannot find module '…/environments/environment'	Ruta relativa incorrecta en el `import` del servicio.	Desde `features/categorias/services` son cuatro niveles: `../../../../environments/environment`.

4. Referencias

Angular. (s. f.-a). *Components*. https://angular.dev/guide/components

Angular. (s. f.-b). *HTTP client*. https://angular.dev/guide/http

Angular. (s. f.-c). *Reactive forms*. https://angular.dev/guide/forms/reactive-forms

Angular. (s. f.-d). *Routing overview*. https://angular.dev/guide/routing

Angular. (s. f.-e). *Signals overview*. https://angular.dev/guide/signals

Bampakos, A., & Deeleman, P. (2023). *Learning Angular* (4.ª ed.). Packt Publishing.

MDN Web Docs. (s. f.). *Cross-Origin Resource Sharing (CORS)*. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

Spring. (s. f.). *CORS*. Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/web/webmvc-cors.html